

HelixLLM

HelixLLM

Один бинарный файл, шесть режимов — совместимый с OpenAI и Anthropic инференс от ноутбука до многохостового кластера.

Краткое описание

HelixLLM — это корпоративная распределённая система LLM на базе Go: единый бинарный файл с системой режимов, масштабируемой от локальной разработки до промышленной эксплуатации на нескольких хостах. Она предоставляет полностью совместимые с OpenAI и Anthropic API через HTTP/3, поддерживает локальный инференс через llama.cpp, цепочку отказоустойчивых провайдеров с оценкой качества, конвейер RAG и систему агентов ReAct.

Краткая аннотация

HelixLLM — это распределённая система LLM на базе Go, реализованная в виде единого бинарного файла. Она предоставляет API, совместимые с OpenAI и Anthropic, через HTTP/3, выполняет локальный инференс с помощью llama.cpp, автоматически обнаруживает и оценивает бесплатные облачные провайдеры для построения цепочки отказоустойчивости, а также включает конвейер знаний RAG и агента ReAct с поддержкой вызова инструментов — и всё это развёртывается в шести режимах.

Подробное описание

HelixLLM — это корпоративная распределённая система LLM, построенная на базе Go с использованием Gin. Её ключевая особенность заключается в том, что один артефакт обслуживает все сценарии масштабирования. Она компилируется в единый бинарный файл, режим работы

которого определяется на этапе развёртывания: запустите его как `full` для полнофункционального экземпляра на ноутбуке или распределите обязанности между режимами `gateway`, `brain`, `knowledge`, `agents` и `control`, разнесёнными по нескольким хостам. При этом используется один и тот же код, лишь перераспределяемый, а не переписываемый, — от рабочей станции разработчика до промышленного кластера.

Система свободно поддерживает два диалекта: полностью совместимые с OpenAI и Anthropic API, благодаря чему существующие клиенты SDK из любой экосистемы работают без изменений. Всё это обслуживается через HTTP/3 (QUIC) с автоматическим резервированием на HTTP/2 и поддержкой TLS 1.3. Локальный инференс выполняется через `llama.cpp` с поддержкой CUDA, Metal и ROCm, что позволяет ускорять работу на оборудовании Nvidia, Apple и AMD. Особенностью является цепочка отказоустойчивых провайдеров, которая превращает известную ненадёжность бесплатных облачных инференс-сервисов в управляемый самовосстанавливающийся ресурс: HelixLLM автоматически обнаруживает бесплатные модели от 7+ облачных провайдеров (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), оценивает их качество по метрикам LLMsVerifier с обновлением каждые 5 минут и маршрутизирует запросы по ранжированной цепочке с автоматическим переключением при ошибках 429/5xx. При этом локальный `llama.cpp` всегда остаётся гарантированным запасным вариантом, так что запрос никогда не завершится неудачей из-за отсутствия доступного провайдера.

Помимо базового инференса, HelixLLM представляет собой полноценную платформу для приложений: в том же бинарном файле реализованы конвейер знаний RAG (индексация, разбиение на фрагменты, векторизация, поиск vector) и система агентов ReAct с поддержкой вызова инструментов, сессий диалога и интеграцией с RAG. Система режимов проявляет свои преимущества и на уровне взаимодействия: в режиме `full` все компоненты общаются через прямые внутрипроцессные вызовы Go без сетевых накладных расходов, тогда как тот же бинарный файл, распределённый по хостам, координирует работу через gRPC, SSE и Kafka. Дополняют картину поддержка согласования содержимого Brotli/gzip, потоковая передача SSE с полным соответствием форматов OpenAI и Anthropic на уровне байтов, аутентификация по ключам API и JWT с ограничением частоты запросов, метрики Prometheus, трассировка OpenTelemetry и обширный набор производственных модулей Go.

Почему мы его создали

Командам нужна переносимая, совместимая со стандартами и устойчивая система инференса — без переписывания клиентов и зависимости от одного провайдера или одной машины. HelixLLM был разработан так, чтобы один и тот же бинарный файл мог работать локально на этапе разработки и масштабироваться до производственного кластера с несколькими хостами, поддерживая диалекты OpenAI и Anthropic, которые уже используют клиенты.

Почему это меняет правила игры

Он объединяет весь стек инференса — шлюз, локальный инференс, облачный фоллбэк, RAG и агентов — в один бинарный файл, управляемый переключателем режимов. Теперь архитектура, которую вы развёртываете, становится вопросом времени выполнения, а не проектом по смене платформы. При этом то, что раньше было уязвимостью, превращается в преимущество: надёжность облачных провайдеров становится первоочередной задачей, непрерывно отслеживаемой и управляемой системой самовосстановления с ранжированием провайдеров каждые несколько минут. В случае сбоя система всегда переключается на гарантированный локальный инференс. В результате появляется единая конечная точка, на которую можно положиться: совместимая со стандартами, переносимая с ноутбука на кластер и неуязвимая к отключению из-за ограничений по запросам или сбоев у вышестоящего провайдера.

Что здесь нового

- Один бинарный файл с шестирежимной системой, работающий как в монолитном, так и в распределённом варианте: прямые внутрипроцессные вызовы Go в режиме full, gRPC/SSE/Kafka при разделении — топология развёртывания меняется без изменений в коде и без дополнительных сетевых накладных расходов.
- Автоматически обнаруживаемая цепочка фоллбэка между несколькими провайдерами (7+ бесплатных), непрерывно ранжируемая по LLMsVerifier с автоматическим переключением при ошибках 429/5xx и гарантированным резервным вариантом на базе llama.cpp — бесплатные мощности превращаются в надёжные.
- Двойная совместимость с OpenAI и Anthropic через HTTP/3 с автоматическим фоллбэком на HTTP/2, благодаря чему

клиенты из любой экосистемы подключаются без модификаций.

- Локальный инференс, поддерживающий CUDA, Metal и ROCm из одного кодовой базы — один и тот же билд ускоряется на оборудовании Nvidia, Apple и AMD.

Основные технические вызовы и их решения

- **Масштабирование с одного хоста на множество без переписывания кода.** Большинство систем жёстко разделяют «локальную разработку» и «распределённое производство», и переход между ними требует перестройки архитектуры. Мы устранили это разделение с помощью системы режимов в одном бинарном файле: одни и те же слои взаимодействуют через прямые внутрипроцессные вызовы в режиме full и прозрачно переключаются на gRPC/SSE/Kafka в распределённых режимах. Масштабирование теперь — это вопрос конфигурации, а не портирования.
- **Ненадёжные и ограниченные по запросам бесплатные облачные провайдеры.** Бесплатный инференс работает быстро, пока не вернёт ошибку 429 или не отвалится на середине запроса. Мы сделали его надёжным, автоматически обнаруживая доступные модели, оценивая их по LLMsVerifier, активно отслеживая заголовки ограничений по запросам, чтобы заранее перенаправлять трафик от провайдеров, которые вот-вот начнут дросселировать, и автоматически переключаясь на резервные варианты вниз по ранжированной цепочке до локального llama.cpp. В результате нестабильность пула никогда не доходит до вызывающей стороны.
- **Совместимость клиентов в двух экосистемах.** Переписывание клиентов под новый бэкенд инференса — это не вариант. Мы реализовали поддержку как OpenAI, так и Anthropic API, включая их специфические форматы потоковой передачи SSE, чтобы SDK из любой экосистемы могли подключаться к HelixLLM без изменений.

Технологический стек

- **Go + Gin** — выбраны, поскольку однобинарная среда исполнения с приоритетом конкурентности делает возможной всю систему режимов: одна сборка, способная работать как сервер на ноутбуке или роль в кластере. Она включает в себя всю систему и HTTP-слой шлюза.
- **HTTP/3 (QUIC) + TLS 1.3 с резервным HTTP/2** — выбраны для современной низколатентной и устойчивой к

разрывам соединений передачи данных, представленной как серверный интерфейс с автоматической настройкой, чтобы клиенты, не поддерживающие QUIC, незаметно переключались на HTTP/2.

- **llama.cpp (CUDA/Metal/ROCM)** — выбран для портативного локального вывода, ускоряемого на бэкендах Nvidia, Apple и AMD из одной кодобазы; одновременно служит гарантированным резервным провайдером, не позволяющим цепочке откатов когда-либо полностью исчерпаться.
- **LLMsVerifier** — выбран, чтобы превратить вопрос «какой провайдер сейчас лучше» в число; он оценивает и ранжирует облачную цепочку откатов с обновлением каждые 5 минут, чтобы маршрутизация учитывала текущее качество, а не устаревшие предположения.
- **Облачные провайдеры (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together)** — выбраны для использования бесплатных мощностей разных апстримов; автоматически обнаруживаются и ранжируются в единую цепочку отказов, чтобы ни один провайдер не становился точкой отказа.
- **gRPC + SSE + Kafka** — выбраны в качестве транспортных протоколов для распределённых развёртываний: gRPC для межсервисных вызовов, SSE для потоковой передачи, а Kafka — для развязанного потока событий между ролями.
- **Векторное хранилище / embeddings** — выбрано для обеспечения работы конвейера знаний RAG от начала до конца: загрузка, сегментация, векторизация и поиск по документам, на основе которых формируются ответы модели.
- **Prometheus + OpenTelemetry** — выбраны для сбора метрик и распределённой трассировки, сопровождающих запрос независимо от используемых режимов развёртывания.
- **Подмодули vasic-digital Go** — выбраны для повторного использования проверенных примитивов производственной инфраструктуры вместо их пересоздания, что обеспечивает согласованность основ системы с более широким стеком.

Статус и замечания о честности

- **Статус: бета-версия.** Функциональная, активно развивающаяся система распределённого вывода.
- **Лицензия: не определена.** В метаданных репозитория не указана лицензия (licenseInfo — null), что требует проверки перед объявлением лицензии.
- Канонический репозиторий в настоящее время размещён по адресу github.com/HelixDevelopment/llm; путь HelixLLM перенаправляет на него. Показатели покрытия тестами и

количество подмодулей в README указаны на основе
самоотчёта.

Приоритетный уровень: Helix-основной.